



Preliminary Study on Fine-Grained Power and Energy Measurements on Grace Hopper GH200 with Open-Source Performance Tools

Oscar Hernandez
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA
oscar@ornl.gov

Filippo Spiga
NVIDIA Ltd
Cambridge, United Kingdom
fspiga@nvidia.com

Thomas Wang
Camas High School
Camas, Washington, USA
twangapples@gmail.com

Francesca Tartaglione
NVIDIA Corporation
Santa Clara, California, USA
ftartaglione@nvidia.com

Wael Elwasif
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA
elwasifwr@ornl.gov

Markus Eisenbach
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA
eisenbachm@ornl.gov

Ross Miller
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA
rgmiller@ornl.gov

Abstract

The increasing adoption of tightly integrated, heterogeneous architectures, combined with the slowdown of Moore's law, has made application power and energy-driven optimizations critical to efficiently use high-performance computing systems. This paper introduces a newly developed open-source toolkit that seamlessly integrates the Linux real-time hardware monitoring program `hwmmon` with the Performance Application Programming Interface and the Score-P performance measurement system, thereby enabling fine-grained power and energy measurements for high-performance computing applications. Our primary target platform is the Wombat test bed, which is a system based on the NVIDIA GH200 superchip. The toolkit can capture transient power peaks with high temporal resolution (50 ms) and, thanks to Score-P integration, can map power metrics to specific code regions, thereby providing actionable information on power-intensive operations and inefficiencies. The toolkit also provides a holistic view of both the power and the energy consumption of the entire GH200 superchip by covering all major components: the Grace CPU, the Hopper GPU, and the I/O subsystem. Experiments that use Locally Self-consistent Multiple Scattering, which is an application for first-principles calculations of materials developed at Oak Ridge National Laboratory, have demonstrated the tool's ability to identify transient power spikes and uncover opportunities for energy-aware optimizations. Additionally, we introduce a Python-based utility for converting Open Trace Format 2 traces to Parquet format, thus enabling advanced

data analysis for numerical integration methods applied to power data for accurate energy profiling.

CCS Concepts

• **Computing methodologies** → **Simulation evaluation**; **Parallel programming languages**; • **Computer systems organization** → **Parallel architectures**.

Keywords

Power measurements, HPC applications, performance tools, LSMS application, NVIDIA GPUs

ACM Reference Format:

Oscar Hernandez, Thomas Wang, Wael Elwasif, Filippo Spiga, Francesca Tartaglione, Markus Eisenbach, and Ross Miller. 2025. Preliminary Study on Fine-Grained Power and Energy Measurements on Grace Hopper GH200 with Open-Source Performance Tools. In *2025 International Conference on High Performance Computing in Asia-Pacific Region Workshop Proceedings (HPC Asia 2025 Workshops)*, February 19–21, 2025, Hsinchu, Taiwan. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3703001.3724383>

1 Introduction

Power and energy optimizations have become critical for high-performance computing (HPC) systems. This need for optimization is due to the increasing complexity of tightly integrated, heterogeneous HPC architectures—such as the NVIDIA GH200 superchip, which is equipped with an Arm-based Grace CPU and capable of consuming up to 1 kW—and the slowdown of Moore's law. These superchip architectures provide substantial



This work is licensed under a Creative Commons Attribution International 4.0 License.

HPC Asia 2025 Workshops, Hsinchu, Taiwan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1342-2/25/02
<https://doi.org/10.1145/3703001.3724383>

Notice: This manuscript has been authored by UT-Battelle LLC under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<https://www.energy.gov/doe-public-access-plan>).

improvements in CPU-GPU performance through deep integration, complex hierarchies, and dynamic resource management. However, achieving fine-grained optimization of applications to meet specific power and energy consumption goals remains a significant challenge, particularly as HPC and data centers deploy systems with tens of thousands of superchips.

In this paper, we address these challenges by introducing a novel, open-source toolkit that integrates the Linux `hwmon` subsystem with the Performance Application Programming Interface (PAPI) and Score-P to provide fine-grained power and energy measurements of HPC applications. Unlike other monitoring approaches, our framework leverages the high temporal resolution (50 ms by default) of `hwmon` to capture transient power peaks across all major components of the NVIDIA GH200 superchip, including the Grace CPU, Hopper GPU, and I/O subsystem. These measurements are seamlessly integrated with Score-P to map power metrics to specific code regions and are visualized using Vampir, enabling developers to gain detailed insights into the application’s behavior. Our toolkit improves the resolution and accuracy of power measurements and broadens their scope to include the entire superchip and its components, offering a complete view of power consumption. The main contributions of this paper are as follows:

- Development of an extended PAPI component that accesses `hwmon` power readings of the GH200 superchip platform, enabling high-resolution measurements across all superchip components and integration into Score-P.
- Demonstration of the toolkit’s ability to capture transient power spikes, providing actionable insights for optimizing power-intensive GPU kernels and addressing resource underutilization in applications.
- A visual comparative analysis between `hwmon` and the NVIDIA Management Library (NVML), highlighting the benefits of high resolution for sampling transient events that strongly impact the estimation of energy consumption in fine-grained application regions.
- Practical evaluation of the toolkit using LSMS, showing its ability to identify transient power spikes and uncover opportunities for energy- and power-aware optimizations.
- Implementation of a Python-based utility that converts Open Trace Format 2 (OTF2) traces to Parquet format, thereby providing a scalable and efficient format to represent trace data. This enables advanced data analytics tools to estimate energy consumption in fine-grained application regions.
- A first set of postprocessing tools—for example, numerical integration methods (trapezoidal and Simpson’s rule)—within Jupyter notebooks to estimate the energy consumption for fine-grained regions. This approach facilitates accurate energy profiling of specific regions of code, including GPU kernels and library calls, thereby helping to optimize energy.

The remainder of this paper is organized as follows. Section 2 describes the architecture and implementation of our toolkit. Section 3 outlines the experimental setup and the applications used for evaluation. Section 4 presents the results of the power and energy measurements, including transient peak analysis and optimization

opportunities. Finally, Section 6 concludes the paper and discusses future directions.

2 Open-Source Performance toolkit for Power and Energy Studies on NVIDIA GH200 Superchip

Figure 1 provides an overview of the toolkit used for power and energy studies. The toolkit builds upon established open-source tools, extending or complementing their capabilities to address fine-grained power measurement needs. The figure highlights the extended components, which include the CoreTemp `hwmon` PAPI component for measuring GH200 power information, an OTF2-to-Parquet utility, a Score-P-based custom library wrapper to intercept Arm Performance Libraries (ArmPL) calls, and a set of Python programs for calculating fine-grained energy regions using Jupyter notebooks and the Parquet trace format.

For the remainder of this section, we briefly introduce each tool, explain the development of the extensions, and demonstrate their use in fine-grained power measurements and overall energy consumption estimation.

2.1 PAPI

PAPI [16] is a widely used, portable framework for accessing hardware performance counters on diverse CPU and GPU architectures. The PAPI CUDA component is designed to measure hardware performance counters specifically for the NVIDIA CUDA platform, enabling access to the GPU’s internal hardware counters. This component leverages the CUDA Profiling Tools Interface (CUPTI) [26] support integrated into the NVIDIA driver library and the NVIDIA HPC Software Development Kit. In environments with the CUPTI-enabled driver installed, the PAPI CUDA component offers comprehensive performance counter data related to GPU kernel execution. This technology allows developers and researchers to gain valuable insights into the performance characteristics of their CUDA-based applications, facilitating the optimization and fine-tuning of GPU-accelerated code. Additionally, the PAPI `nvml` component leverages NVML [25], which provides direct access via API to GPU power monitoring, frequency tracking, and dynamic power capping.

Our work extends PAPI’s CoreTemp [33] component by integrating data from the Linux `hwmon` interface, enabling access to fine-grained power metrics for the GH200 superchip platform. This extension provides seamless interoperability with higher-level tracers and profilers.

2.2 Score-P

Score-P [18] is a scalable and open-source performance measurement tool designed to provide profiling, event trace recording, and online analysis of HPC applications. It uses OTF2 [12] and supports multiple parallel programming paradigms, including Message Passing Interface (MPI), OpenMP, CUDA, HIP, and SHMEM. Score-P can be used with high-level visual tools such as Vampir [34], Cube [27], Scalasca [14], and Tuning and Analysis Utilities (TAU) [30].

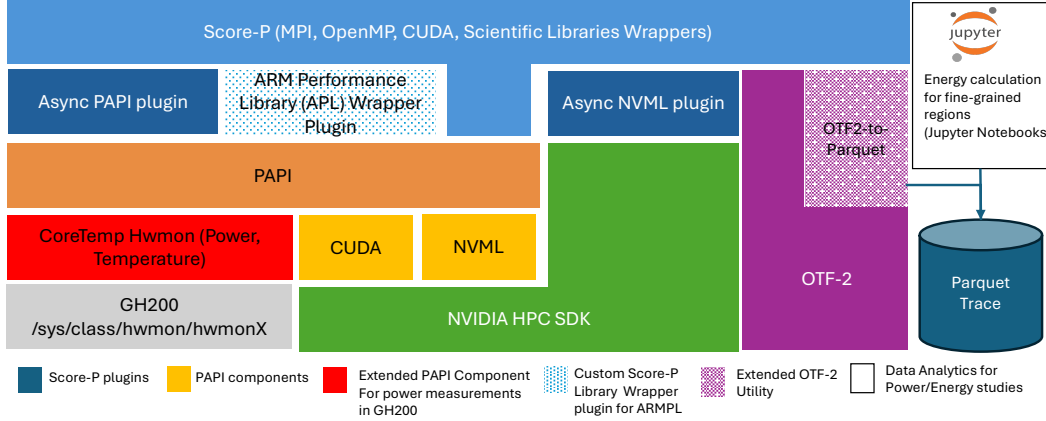


Figure 1: Software stack diagram that illustrates the integration of Score-P, PAPI, the extended hwmon component, and the OTF2-to-Parquet conversion utility for fine-grained power measurements and energy estimation on the GH200 platform.

A key strength of Score-P is its plug-in infrastructure, which enables integration with external metric monitoring sources available via PAPI components or vendor-specific APIs. This functionality is leveraged in the Async PAPI (APAPI) plug-in, which asynchronously samples hardware counters using an additional monitoring thread while minimizing application measurement overhead. The Score-P NVML plug-in uses NVML to access NVIDIA-specific metrics, including GPU power, temperature, and utilization. It uses the `nvmlDeviceGetPowerUsage()` API.

Our proposed enhanced PAPI extends Score-P capabilities by enabling mapping of power metrics collected via hwmon to specific regions of the application code. Thus, developers can visualize power and estimate energy consumption at the trace level using a tool such as Vampir and correlate that data with key performance events and code regions, providing actionable insights for power- or energy-aware optimizations.

2.3 Power Measurement Support via hwmon on GH200 Superchip

The Linux hwmon subsystem provides a standardized interface to access hardware sensors. Originally designed for thermal monitoring, hwmon now includes support for power, energy, and other critical node-level metrics for system optimization on a variety of architectures. Sensor values are exposed through the file system under `/sys/class/hwmon/`, enabling a standard way to integrate with performance analysis tools. On the NVIDIA GH200 superchip, hwmon exposes power sensors that monitor critical components. The *NVIDIA Grace Performance Tuning Guide* [24] provides a diagram of the GH200 superchip telemetry sensors. The available telemetry sensors include **Sensor-0** (total module power of the superchip), **Sensor-1** (Grace CPU power, including DRAM and in-package voltage regulator power), **Sensor-2** (CPU socket power, including in-package voltage regulator power), and **Sensor-3** (out-of-core system I/O power, including in-package voltage regulator power).

2.4 Estimating GPU and High-Bandwidth Memory Power Utilization

The power consumption of the Hopper GPU and its high-bandwidth memory (HBM) can be estimated by subtracting Sensor-1 (Grace CPU power) from Sensor-0 (total module power). This calculation captures the combined power usage of the GPU, the HBM, and its regulators. Although useful, this estimate requires careful interpretation due to the inclusion of its power overhead in the package voltage regulator [23].

Our toolkit can compare hwmon and NVML-based measurements to evaluate their accuracy. Although NVML provides convenient access to GPU power metrics, its API, `nvmlDeviceGetPowerUsage()`, adds a 1 s averaging interval, which introduces delays that can cause underreporting of transient power spikes. In contrast, hwmon offers a higher temporal resolution (50 ms by default), enabling more precise mapping of power usage to performance events for the GPU and the other components of the superchip. Table 1 highlights the differences between hwmon and the NVML approach.

Existing tools for performance and power analysis, such as NVIDIA’s Data Center GPU Manager and Nsight Systems [8], have traditionally focused on providing GPU average power metrics, often relying on coarse-grained running average intervals. For example, the function `nvmlDeviceGetPowerUsage()` [25] used by Nsight Systems and the Score-P NVML plug-in provides a running average of 1 s of GPU power consumption. Although this approach effectively smooths out short-term power fluctuations, it limits the ability to capture transient power peaks, which are critical for understanding fine-grained power and energy behavior in application code regions. This limitation complicates the precise mapping of power usage and energy estimations to specific computational phases, potentially obscuring insights into energy efficiency and application-level optimizations. Although NVML also enables reading instant power via the `NVML_FI_DEV_POWER_INSTANT` API, performance and monitoring tools tend to prioritize stability over granularity in power reporting. Studies [35] have shown that even with millisecond-scale sampling intervals ranging from 20 to 100 ms (depending on the GPU architecture), a significant portion of

Table 1: Comparison of hwmon and NVML for power monitoring on GH200 superchip

Feature	hwmon	nvmlDeviceGetPowerUsage()
Scope	Superchip, Grace CPU, SysIO, GPU	GPU-only metrics
Sampling interval	50 ms (default, configurable)	20–100 ms depending on the GPU architecture [35]
Granularity	High-resolution power spikes	Averaged power over time (1 s)
Accessibility	File-based system in Linux	NVML API
Flexibility	Support for dynamic interval adjustment	Fixed interval
Overhead	Low I/O overhead	Minimal API overhead
Use case	Fine-grained event correlation	Coarse-grained monitoring

run-time activity remains unsampled. For example, tools such as `nvidia-smi` may capture only a fraction (20%–25%) of the runtime at this resolution, leading to substantial underreporting of energy consumption—up to 68.6% in certain scenarios.

2.5 Extending PAPI with hwmon GH200 Power Readings

The CoreTemp PAPI component, initially designed for thermal monitoring using `hwmon`, was extended to support power measurement on the GH200 platform. Key improvements include (1) the addition of `power_average`, `power_min`, and `power_max`, providing detailed power measurements in microwatts; (2) improved parsing logic to robustly handle deeply nested file structures as well as updated event insertion processes with detailed descriptions and units; and (3) efficient enumeration of up to 32 sensors per metric type and error recovery mechanisms to ensure continuous operation even when some sensor files are missing or inaccessible.

2.6 OTF2-to-Parquet Conversion Utility

The Score-P measurement system uses the OTF2 format, which is designed to handle scalable performance data for HPC tools. However, OTF2 is not widely known outside the HPC community. Data scientists typically work with formats that are better suited for analytics and machine learning. Calculating fine-grained energy metrics requires using derived metrics available in tools (e.g., Vampir, Cube) that work with OTF2. However, these tools might not have all the necessary features, such as advanced integration methods, better data analysis options, or support for machine learning workflows. Additionally, changing Score-P to output data in a different trace format would require much engineering work, affect the way events are collected at runtime, and potentially break compatibility with performance tools that use OTF2 traces.

As part of our open-source toolkit, we developed a postmortem, Python-based utility—`otf2parquet.py`—to convert OTF2 trace data into Apache Parquet format [10], facilitating efficient and scalable trace representation for data analysis tools. This utility processes OTF2 archives to extract both event data and definitions, transforming them into structured Parquet files compatible with modern data analytics frameworks such as Pandas, SciPy, and NVIDIA RAPIDS. By iterating over trace definitions and events, the program captures relevant metadata, including region entries and exits, and ensures compatibility by converting nested or complex structures into a flat, tabular format. This enables fine-grained analysis of HPC application traces using powerful data processing tools.

The utility processes OTF2 trace data into structured Parquet files, thereby ensuring schema consistency and enabling scalable analytics. It outputs separate Parquet files for definitions and events, and those files can be used in Python programs for detailed investigations of performance metrics and energy consumption at scale. This enables experimentation with custom-derived metrics, allowing users to develop energy-based figures of merit for their applications.

2.7 Data Analytics for Fine-Grained Region Energy Estimation

To better estimate the actual energy consumption of fine-grained regions in the code from collected trace, we developed a postprocessing tool that implements numerical integration methods based on the trapezoidal rule and Simpson’s rule [5] using Pandas, SciPy, and Matplotlib within Jupyter notebooks. The *trapezoidal rule* draws a line between two data points and then evaluates the area in the trapezoid. Similarly, *Simpson’s rule* draws a quadratic equation to fit three points and then uses simple algebra to find the area under a curve. In our case, the sampling frequency is so fine that both methods yield extremely similar results (within 0.02% of each other), so we use the trapezoidal method due to less overhead for a slightly shorter runtime. To implement these methods, we mapped power samples to *exclusive* and *inclusive* performance regions of the code such as GPU kernels, function or library calls, and parallel loops. Exclusive regions correspond to segments in which a specific function or routine is executed, whereas inclusive regions include the function and all its nested calls. This mapping was derived from the tracing data, combining the power samples with the corresponding time intervals in the code regions. Using the calculated areas under the power curve for each region, we estimated the energy consumption for important regions in the code, enabling the identification of energy hot spots and the optimization of specific regions for energy efficiency.

2.7.1 Detailed Implementation for Fine-Grained Region Energy Estimation. The detailed implementation of fine-grained energy estimation involves preprocessing the Parquet trace data, mapping events to regions, calculating energy with numerical integration, and visualizing the results. The following Python snippets show the core implementation steps.

The program begins by reading the Parquet files generated by the `otf2parquet.py` utility, which include separate Parquet files for OTF2 events and definitions. The *event* file contains trace events, such as Enter and Leave events with associated time stamps

and metrics (e.g., hardware counters, power measurements). The *definition* file provides metadata on regions, metrics, and their structures. These Parquet files are loaded using Pandas. Listing 1 shows how both files are read and preprocessed for analysis.

The time stamps are converted from nanoseconds to seconds to use standard energy calculations. This preprocessing ensures that the data is ready for numerical integration and region mapping. Listing 2 shows how the program preprocesses power data by converting the value column to numeric data, handling invalid entries, and removing missing values. Power metrics are converted to standard units (e.g., watts) using the appropriate scaling factor because NVML power values are reported in milliwatts and hwmon values in microwatts.

```
1 import pandas as pd # For data manipulation
2 import numpy as np # For numerical operations
3 from scipy.integrate import trapezoid # For numerical integration
4
5 # Read the Parquet files containing event and definition traces
6 events_file = "trace_events.parquet"
7 definitions_file = "trace_definitions.parquet"
8
9 # Load event data
10 df_events = pd.read_parquet(events_file,
11                             columns=["time", "metric",
12                                     "values", "region"])
13
14 # Ensure time is numeric
15 df_events["time"] = pd.to_numeric(df_events["time"])
16 # Convert to seconds
17 df_events["time"] = (df_events["time"] -
18                     df_events["time"].iloc[0]) / 1e9
19
20 # Load definition data
21 df_definitions = pd.read_parquet(definitions_file)
```

Listing 1: Loading events and definitions Parquet files

```
1 def preprocess_power_data(df_power, metric_scale):
2     df_power['values'] = pd.to_numeric(df_power['values'],
3                                       errors='coerce')
4     df_power.dropna(inplace=True)
5     df_power['values'] /= metric_scale # Scale metrics (e.g.,
6                                         from microwatts to watts)
7     return df_power
```

Listing 2: Preprocessing power data

Listing 3 demonstrates the mapping of events and the calculation of energy using the trapezoidal rule.

```
1 from scipy.integrate import trapezoid
2
3 def calculate_energy(df_events, df_power):
4     energies = []
5     for _, enter_event in df_events[df_events['event'] == 'Enter'].
6         iterrows():
7         region = enter_event['region']
8         enter_time = enter_event['time']
9         leave_time = df_events[(df_events['event'] == 'Leave') &
10                                (df_events['region'] == region) &
11                                (df_events['time'] > enter_time)] \
12                                ['time'].min()
13
14         if not pd.isna(leave_time):
15             power_data = df_power[(df_power['time'] >= enter_time) &
16                                   (df_power['time'] <= leave_time)]
17             energy = trapezoid(power_data['values'],
18                               power_data['time'])
19             energies.append({'region': region, 'energy': energy,
20                             'duration': leave_time - enter_time})
21     return pd.DataFrame(energies)
```

Listing 3: Mapping events and calculating energy

Finally, energy consumption is plotted with Matplotlib to display energy-intensive regions and run-time information (Listing 4).

```
1 import matplotlib
2 import matplotlib.pyplot as plt
3
4 def plot_energy_by_region(df_energy):
5     df_energy_sum = df_energy.groupby('region')['energy'].sum()
6     df_energy_sum = df_energy_sum[df_energy_sum > 1.0] # Filter
7     # Significant energy use
8     df_energy_sum.sort_values(ascending=True).plot(kind='barh',
9             figsize=(10, 6), color='skyblue')
10     plt.xlabel('Energy Consumption (J)', fontsize=12)
11     plt.ylabel('Region Name', fontsize=12)
12     plt.title('Energy Consumption by Region')
13     plt.tight_layout()
14     plt.show()
```

Listing 4: Visualizing energy by region

2.8 Measurement and Analysis Methodology

Using the tool set involves several steps that seamlessly integrate fine-grained power measurements and energy analysis. The process begins with manual or automatic code instrumentation using tools such as Score-P, which enables the collection of detailed tracing points for both performance and power metrics. The instrumented application is then executed, during which tracing points are measured to provide insights into the behavior of the application's source code regions and the power information retrieved from hwmon via PAPI and the NVML Score-P plug-in. The next step involves visualizing the collected trace data using tools such as Vampir, which helps in understanding runtime, power consumption, and accumulated energy behavior across different application regions. Following visualization, the trace data in OTF2 is converted into Apache Parquet files using our Python OTF2-to-Parquet conversion utility, facilitating advanced data analytics on the trace data. Finally, we use SciPy to employ numerical integration techniques, such as the trapezoidal and Simpson's rules, to estimate energy consumption and identify optimization opportunities across fine-grained code regions.

3 Experiments and Evaluation

For our power and energy experiments, we used the Locally Self-consistent Multiple Scattering (LSMS) application on a GH200 testbed system. The following sections describe LSMS, the testbed system, and how the toolkit was used for our power and energy studies.

3.1 Testbed System

At the Oak Ridge Leadership Computing Facility, we used an eight-node cluster made of Supermicro ARS-111GL-NHR systems called *Wombat* to evaluate our toolchain. Each node used Red Hat Enterprise Linux 9.4 with kernel version 5.14.0-427.37.1.el9_4.aarch64+64k. The nodes contained a single NVIDIA GH200 superchip [32] with a 72-core Arm Neoverse-V2 CPU and 480 GB LPDDR5X CPU memory as well as a Hopper GPU with 96 GB HBM3 GPU memory. The NVIDIA driver version was 555.42.02 (CUDA 12.5). For the experiments, we used NVHPC 24.9 and GNU 11.4.1 (provided as default with the Linux distribution). For CPU BLAS and LAPACK library calls, we used the thread-safe version of ArmPL 24.10.0. As part of our toolkit, we used Score-P 8.4 built with APAPI, NVML, and a custom library wrapper plug-in for ArmPL. PAPI 7.0.0 was modified to incorporate

our extended CoreTemp component to measure power telemetry. The GH200 superchip was configured to use up to 1,000 W, with the Grace CPU plus memory having a maximum thermal design power (TDP) of 250 W.

3.2 LSMS

The LSMS exascale application [4, 11] is a density functional theory code used to calculate the electronic properties of materials, focusing on metals, alloys, and nanostructures. LSMS employs the Korrington-Kohn-Rostoker method, which efficiently computes electronic interactions by calculating Green’s function instead of diagonalizing the Hamiltonian. LSMS can achieve near-linear scaling, enabling simulations of up to 100,000 atoms. The computationally intensive operations involve dense-matrix computations, such as multiplications and block inversions, which dominate runtime. To iteratively refine the electron density and potential, LSMS employs a self-consistent field (SCF) loop. LSMS is written in C++ and Fortran. Scalability is achieved using the MPI for atom-level parallelization, OpenMP CPU for energy-level computations, and CUDA/HIP for dense linear algebra operations on GPUs, enabling near-linear scaling on large supercomputers [4].

3.3 Experimental Setup

To evaluate our framework, we built LSMS with Score-P, enabling OpenMP, CUDA, and the ArmPL library wrapper plug-in while disabling automatic compiler instrumentation to reduce overhead and trace file sizes. The setup included one MPI rank, one OpenMP thread (OMP_NUM_THREADS=1), and one GPU for execution. Power tracing was performed using the Score-P apapi_plugin and the nvml_plugin, both sampling at 5 ms intervals. The apapi_plugin monitored GH200 sensors (hwmon[1–4]), whereas the nvml_plugin captured NVML GPU metrics such as power, streaming multiprocessor and memory clocks, usage, PCIe activity, temperature, and utilization. This configuration balanced detailed measurements with manageable overhead, allowing a precise analysis of the energy and performance of LSMS on the GH200 system. For our LSMS runs, we adjusted the *Iron-Platinum* benchmark case¹ to achieve high GPU utilization. The OTF2 traces were visualized with Vampir traces and converted to the Parquet format using our otf2parquet.py utility for fine-grained energy analysis. Although we used this setup to study the LSMS application, the tool can trace power using multiple MPI ranks and OpenMP threads across nodes using the Score-P measurement system.

4 Results and Discussion

4.1 Overall Performance, Power, Accumulated Energy, GPU Utilization, and GPU Temperature

Figure 2 presents the LSMS execution timeline for two SCF iterations, which took a total of 151.942 s and consumed 93.23 kJ of energy. The timeline displays the entire GH200 superchip power and the accumulated energy consumption, along with GPU utilization and temperature during critical GPU kernels and application synchronization phases. During the GPU kernel launch phase,

which includes the scattering zgemm matrix computation, peaks in GPU utilization and power consumption are observed, reflecting sustained activity in GPU kernels. This behavior is characteristic of the computational intensity of dense-matrix operations, such as multiplications and block inversions, which dominate LSMS’s power usage. In contrast, synchronization phases between SCF iterations, where CPU computations dominate, exhibit reduced GPU utilization and power consumption levels. These transitions, visible as drops in power and gradual increases in energy, highlight opportunities to optimize workload distribution between CPU and GPU resources. For example, overlapping CPU and GPU computations wherever possible can reduce idle times, or using dynamic scheduling techniques to redistribute tasks based on power or energy utilization. Figure 2, trace F, also shows how the temperature increases during the first SCF iteration, eventually reaching a steady-state thermal behavior in the next iteration.

4.2 Power Breakdown of the GH200 Superchip

Figure 3 illustrates the breakdown of power consumption of the NVIDIA GH200 superchip in terms of its GPU and CPU, highlighting the differences between the measurements obtained using the Linux hwmon subsystem (trace B) and the Score-P NVML plug-in (trace C). The traces show the maximum (dark red), average (red), and minimum (blue) values for each pixel point in the display, as well as the overall average represented by a horizontal green line.

4.2.1 GPU Power Estimation via hwmon. Total GPU power utilization was estimated by subtracting the power measured by hwmon Sensor-1 (Grace system on chip plus CPU memory power) from Sensor-0 (total module power). This estimation captures the GPU and HBM power, including the regulator power. The readings from the hwmon sensors demonstrated higher power values on average compared to those reported by NVML. Note that both power trace values are averaged at the pixel level of the trace display. This discrepancy arises due to the finer-grained sampling resolution of hwmon and GPU and HBM power regulator power usage, which requires a 50 ms average interval compared to NVML’s 1 s running averages for power measurements. When inspecting the trace data, we observe that the NVML minimum and maximum values are 165 and 606 W, respectively, whereas the hwmon minimum and maximum values are 101 and 789 W, respectively. From Figure 3 trace B, we observe that hwmon captures transient power spikes more effectively, resulting in a wider range of maximum (dark red) and minimum (blue) values, including higher reported values during periods of intense GPU activity and lower values when the GPU is idle.

4.2.2 Grace CPU Power Utilization. The power consumption of the Grace CPU plus memory, monitored using hwmon Sensor-1, remained consistently low throughout the experiments, averaging 48 W, which represents 19% of the total Grace and memory TDP. This is attributed to the LSMS application configuration, which utilized only a single OpenMP thread of the 72 available cores on the Grace CPU. The low utilization of the CPU reflects the workload’s GPU-centric nature and highlights the potential for further optimization when scaling CPU-intensive computations as we increase our problem size.

¹lmax=7, rLIZ=18, rsteps=89.5, 91.5, 93.2, 99.9, atom="Fe", and Z=26

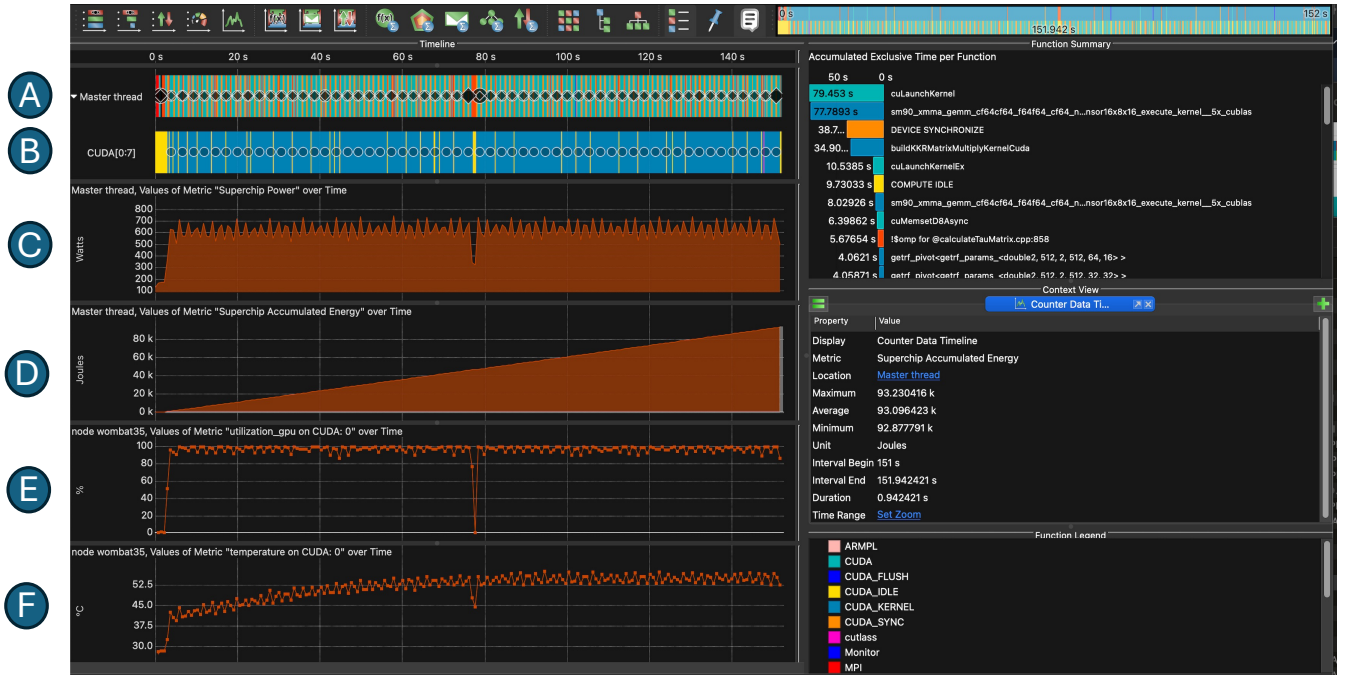


Figure 2: Overview of power, accumulated energy, GPU utilization, and temperature metrics during two SCF iterations in LSMS. Key regions of the code are annotated to highlight computational phases and optimization opportunities. Traces A–F are as follows: A represents the CPU master OpenMP thread managing the SCF iteration loop and offloading kernels to GPU streams. B shows GPU kernels running on streams 0–7, highlighting key regions such as `buildKKRMatrixMultiply` and `cuBLAS zgemm`. C displays power samples of the GH200 superchip in watts. D captures total accumulated energy in joules. E shows GPU utilization. F displays GPU temperature and thermal behavior.

4.2.3 NVML GPU Power Measurements. As discussed in Sections 2.1 and 2.4, the NVML plug-in uses the `nvmlDeviceGetPowerUsage()` function, which reports power consumption using a running average of 1 s [25]. Although this method is well suited for coarse-grained monitoring, the averaging process smooths out transient power spikes, limiting its effectiveness for capturing short-lived events such as GPU kernel launches.

Figure 4 highlights this limitation by comparing the NVML power traces (trace C) with the high-resolution measurements from `hwmon` (trace D). The delayed response of NVML obscures the rapid fluctuations in power, complicating the precise mapping of power usage to specific GPU activities. In contrast, `hwmon` captures these dynamics with finer granularity, providing critical insights into the power behavior of transient GPU workloads.

Although NVML’s power averaging has limitations, its GPU utilization metrics add an important perspective to understanding GPU activity, as shown in trace E. These metrics highlight periods of high GPU usage during dense kernel computations and idle phases when CPU-bound tasks take over. Combining data with GPU power readings and performance events from specific code regions provides a clearer picture of GPU performance and power usage.

During the period between SCF iterations, distinct power profiles reveal the interplay between CPU and GPU activities. These transitions are marked by variations in power consumption as the CPU alternates between launching GPU kernels and executing ArmPL library calls (`getrf`, `getri`, and `zaxpy`), while the GPU enters an idle state. Such power dynamics highlight the need for fine-grained measurement tools to accurately characterize transitions between application phases, enabling optimization such as overlapping new computations during these periods.

4.2.4 Findings from Fine-Grained Power and Energy Tracing. Figure 5 provides a view of the high-resolution trace of power samples associated with individual GPU kernels, providing detailed information on power dynamics during application execution. The trace reveals distinct power profiles for different kernels, which is intuitively expected. For instance, `buildKKRMatrix` consistently consumes less power compared to the `cuBLAS zgemm` library call, primarily because it is memory-bound dominant. Within `cuBLAS zgemm`, individual subkernel executions demonstrate varying power and performance characteristics depending on the kernel inputs, matrix shape, and data-type precision.

The combined analysis of `hwmon` and NVML readings highlights the need for high-resolution power samples to measure the power and accumulated energy consumption of key kernels. Transient power spikes, accurately captured by `hwmon`, align closely with compute-intensive GPU kernel executions, such as dense-matrix

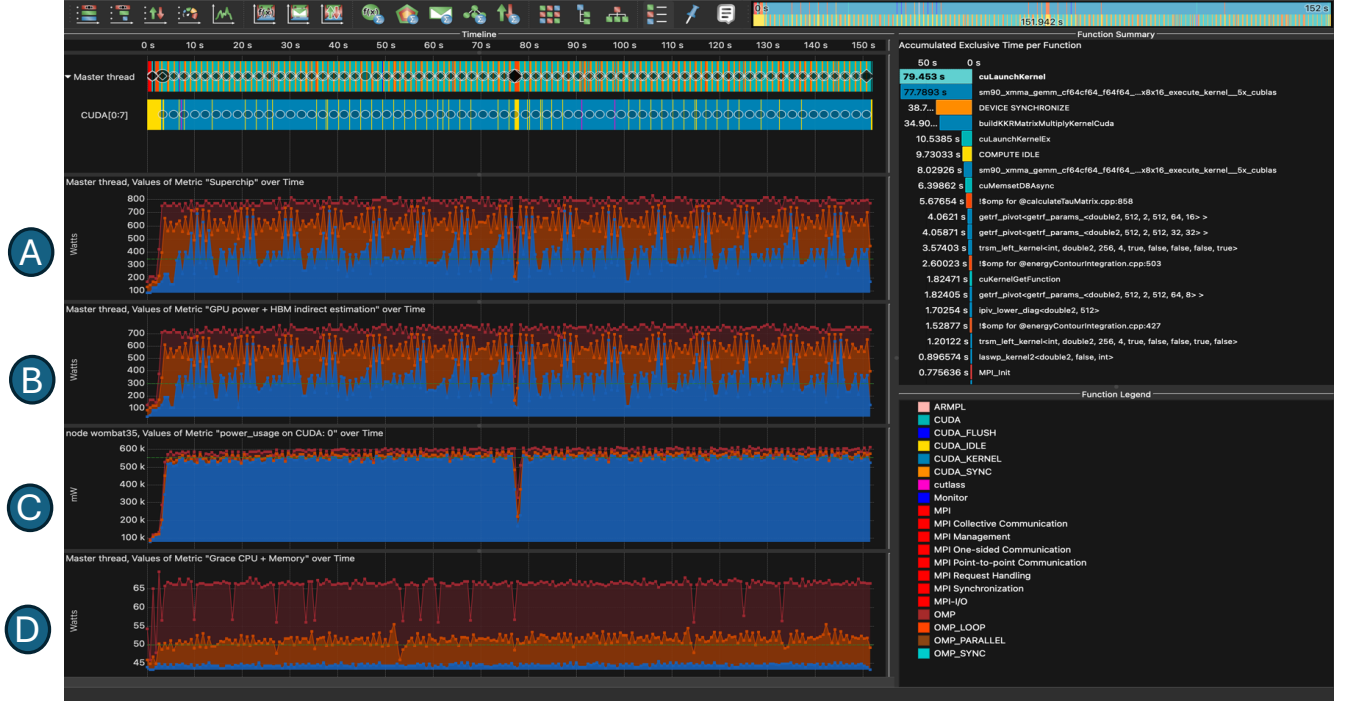


Figure 3: Overview of the power breakdown of the GH200 superchip in terms of GPU and CPU power utilization. Traces A–D are described as follows: A: Power samples of the GH200 superchip in watts, measured using hwmon Sensor-0. B: Estimated GPU power consumption of the GH200, calculated by subtracting the power measured by hwmon Sensor-1 from Sensor-0 in watts. C: GPU power samples measured by NVML in watts. D: Power samples of the Grace CPU and memory, measured by hwmon Sensor-1 in watts. The traces show the maximum (dark red), average (red), and minimum (blue) values for each pixel point in the display, as well as the overall average represented by a horizontal green line.

operations, emphasizing the importance of high-resolution power measurements.

After converting the OTF2 trace to Parquet, we used a Jupyter notebook described in Section 2.7.1 to estimate the energy (in joules) consumed by the most time-consuming GPU regions (e.g., kernels) of LSMS executed on the GH200 superchip. We noticed that the NVML computed average affects the estimated total energy consumption of finer-grained regions or regions with drastic power transitions.

Figure 6 provides a detailed comparison of the energy estimates for the most time-consuming GPU regions during LSMS execution, showing both the NVML (Figure 6a) and hwmon (Figure 6b) measurement methods. This comparison highlights critical differences in the accuracy and granularity of the energy data, thereby providing opportunities for optimization and analysis refinement. For example, the cuBLAS `zgemm` kernel with a tile size of $2 \times 2 \times 1$ has a total runtime of 8.03 s with 94,208 invocations, averaging 85 μ s per call. The energy estimation using NVML (Figure 6a) is 4,519.03 J. This method overestimated the total energy consumption by 10% compared to that of the hwmon method (Figure 6b), which is 4,059.86 J. This discrepancy is primarily attributed to NVML’s reliance on a 1 s averaging interval, which smooths out transient power spikes during dense-matrix operations. Transient power spikes during

GPU kernel execution were captured due to the hwmon resolution, unlike the NVML averaged metric.

Another significant finding relates to the `COMPUTE_IDLE` region, where the GPU enters a low-power state due to the absence of active kernel execution. Here, the NVML method overestimated energy consumption by 26%, underscoring its inability to accurately measure power drops during idle phases. The hwmon method, with its higher temporal resolution, effectively captures these transitions, providing a clearer representation of energy behavior during inactive periods.

4.3 Optimization Opportunities

By analyzing power and energy profiles and correlating them with performance data, developers can discover a wide range of opportunities to refine algorithms and adapt workloads for improved energy efficiency. For example, utilizing the ArmPL multithreaded library with nested parallelism could help reduce the time spent in synchronization phases between SCF iterations. Figure 6 shows that the power and energy consumption varies significantly between the kernel types. Memory-bound kernels such as `buildKKRMatrix` consume less power than compute-intensive kernels, as seen in its lower energy consumption despite prolonged execution times. This suggests that these regions may benefit from power capping to reduce the frequency used by the GPU streaming multiprocessors.

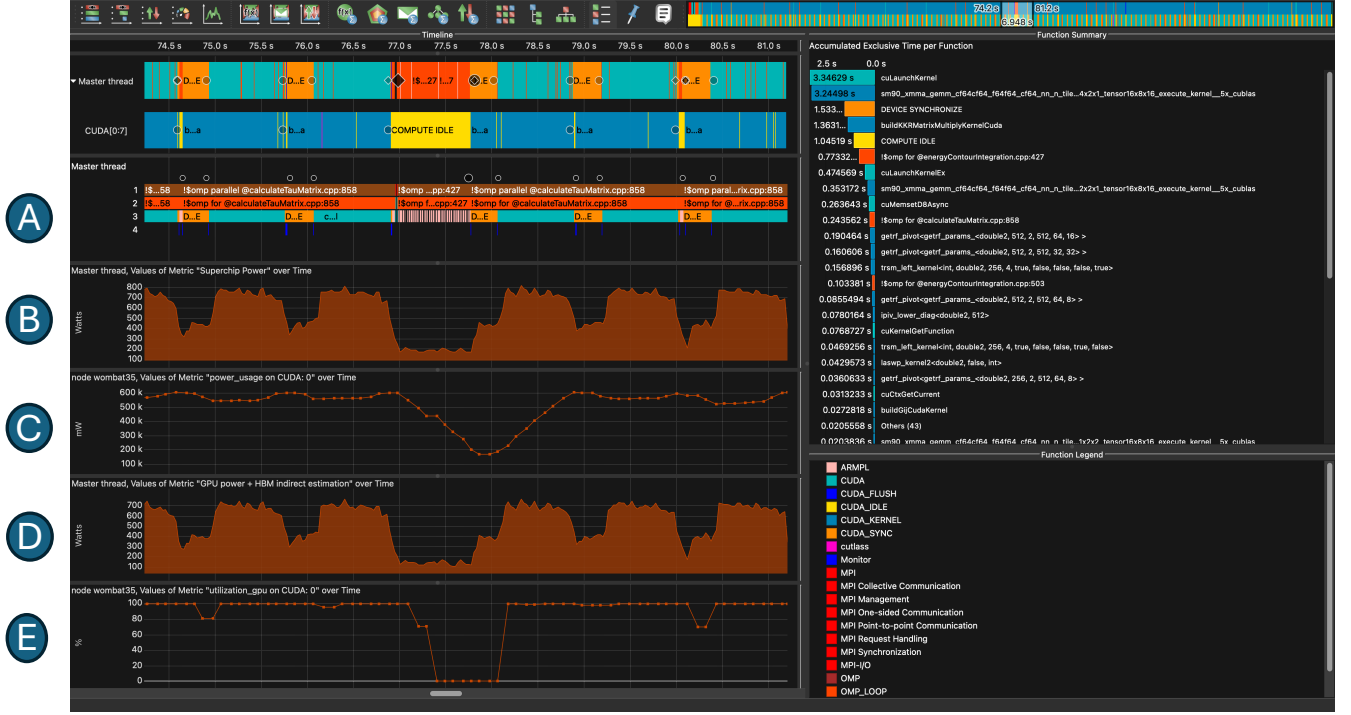


Figure 4: Power breakdown of the GH200 superchip in terms of GPU and CPU power utilization during the transition between SCF iterations. Traces A–D are described as follows: **A** shows the CPU master thread region activity, including the time the CPU spends launching kernels, and executing OpenMP parallel region, the library calls to the ArmPL library. **B** shows the power samples of the GH200 superchip in watts, measured using `hwmon` Sensor-0. **C** shows the GPU power samples measured by NVML in watts. **D** shows the estimated GPU power consumption of the GH200, calculated by subtracting the power measured by `hwmon` Sensor-1 from Sensor-0 in watts. **E** shows the GPU utilization as measured by NVML.

Other optimizations could include dynamically lowering power states or frequencies when the GPU is idle to minimize unnecessary energy consumption. These strategies represent just a subset of the possibilities, and a deeper analysis of power and performance metrics might reveal further opportunities to optimize the application- and system-level operations.

5 Related Work

Variorum [21] also focuses on portability by providing a unified interface to monitor and manage power across various architectures, including x86, Arm, IBM Power CPUs, and NVIDIA GPUs. It uses APIs such as the Running Average Power Limit (RAPL) [17], IBM Opal, `hwmon`, AMD EPYC System Management Interface, NVML, and `rocm_smi` library. Unlike PAPI, which is more generalized for hardware counters, Variorum targets specialized use cases such as vendor-agnostic energy management, highlighting the need for adaptable tools in heterogeneous environments. The Power Measurement Toolkit [6] interfaces with Hewlett Packard Enterprise (HPE) Cray’s `pm_counters`, extracting node-level power and energy metrics, including CPU and memory telemetry. This system-centric approach complements application-level tools by connecting application performance with telemetric data. LIKWID is a suite of command-line utilities designed for measuring performance and

energy efficiency by accessing low-level hardware counters and system libraries and has been extended to support GPUs [19].

Tools such as TAU [29] and Score-P [18] and its plug-in infrastructure [28] are designed for detailed run-time profiling, tracing, and power analysis. The Tracing Heterogeneous APIs infrastructure [13] enables the interception and contextual mapping of performance events, facilitating integration with programming models and enabling context-specific targeted optimizations (e.g., value based). Vendor-specific tools such as NVIDIA Nsight Systems [8], AMD Omnitrace [2], and Linaro MAP [1] leverage vendor-specific APIs (e.g., NVML, `rocProf`) to provide detailed application-level and call stack-level power and performance metrics. These tools often integrate advanced features such as dynamic power information and fine-grained GPU kernel analysis. HPE’s CrayPat [22] uses PAPI alongside vendor-specific telemetry, offering a balanced approach to power measurement and optimization.

Monitoring frameworks designed to characterize workloads, such as Pika [9], integrate with the SLURM [31] job scheduler to enable system-wide energy tracking. NVIDIA’s Data Center GPU Manager [7] is a monitoring and management framework for multi-GPU and multinode systems. It reports GPU power and energy usage using the NVML functions `nvmlDeviceGetPowerUsage` and `nvmlDeviceGetTotalEnergyConsumption`. Its power metrics are GPU specific at millisecond granularity rather than specific to the

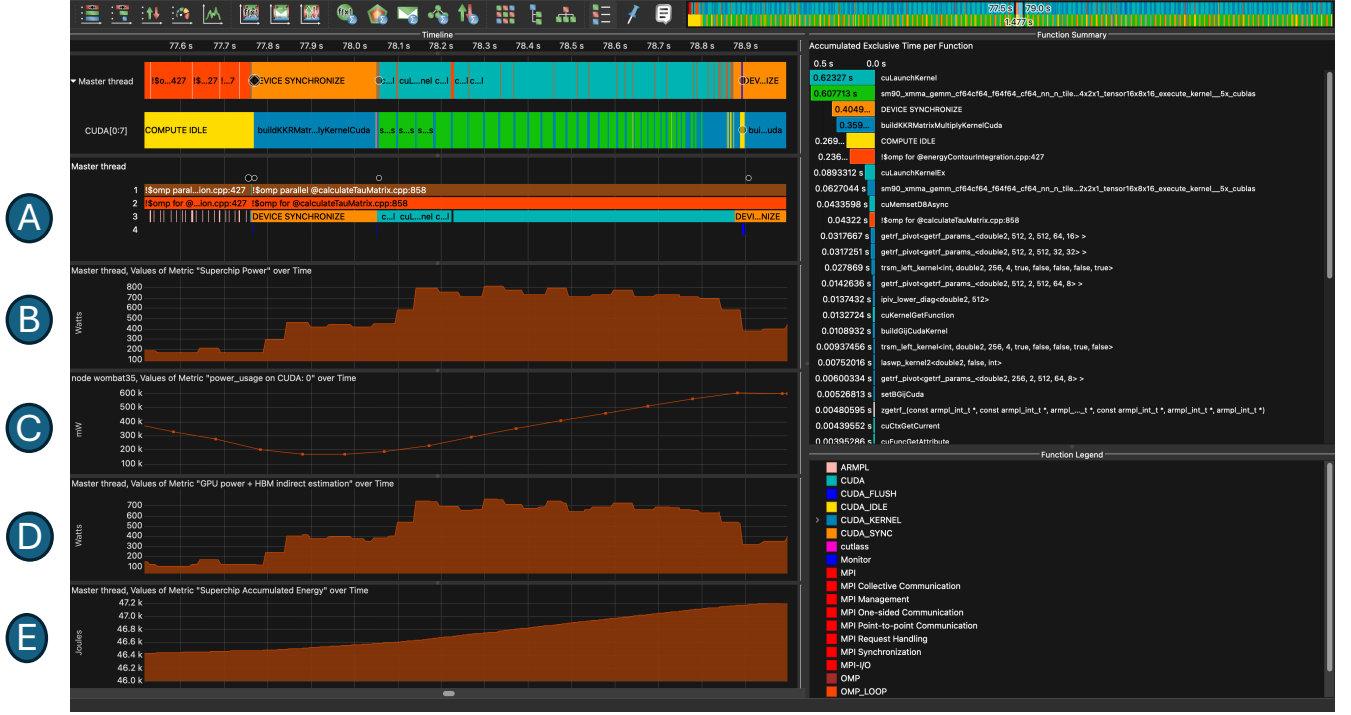


Figure 5: Power breakdown of the GH200 superchip in terms of GPU and CPU power utilization during the transition between SCF iterations. Traces are described as follows: A: CPU master thread activity, including kernel launches and OpenMP parallel regions. B: hwmon power samples for the entire superchip. C: GPU power samples from NVML. D: Estimated GPU power from hwmon Sensor-0 and Sensor-1 subtraction. E: Accumulated superchip energy calculated by integrating hwmon Sensor-0 power values.

entire GH200 superchip. MCBound [3] improves workload optimization by classifying applications as memory or compute bound, facilitating energy-sensitive job scheduling. This integration supports data-driven workload scheduling and power management, which are essential for large-scale HPC deployments.

Emerging fine-grained optimization tools, including the Dynamic Energy-Performance Optimiser [20] and READEX [15], dynamically adjust power caps based on RAPL and NVML metrics, striking a balance between energy efficiency and performance. However, these tools often lack abstractions for programming model events, scalable performance tracing and visualization capabilities, and support for the GH200 superchip.

6 Conclusion and Future Work

We demonstrated a methodology for measuring and analyzing power and energy consumption in production-grade, tightly integrated CPU-GPU compute architectures such as the NVIDIA GH200 superchip. By integrating hwmon-based power readings into PAPI and using Score-P instrumentation alongside Vampir visualization, we provided an effective toolkit for fine-grained power and energy profiling at the code level.

An important contribution of this work is the implementation of an OTF2-to-Parquet conversion utility, enabling scalable and efficient trace representation compatible with modern data analytics frameworks such as Pandas, NumPy, and NVIDIA RAPIDS. This

utility facilitates detailed energy analysis using Jupyter notebooks, employing numerical integration methods such as the trapezoidal rule and Simpson’s rule to enable precise energy estimations for fine-grained application regions. Additionally, it allows users to explore other metrics based on energy or power, providing a flexible framework for data scientists or domain scientists. These capabilities improve the accuracy of energy profiling and uncover optimization opportunities by correlating power metrics with specific computational phases or architecture-specific performance metrics (e.g., hardware counters). We also evaluated energy estimation using the NVML method against the hwmon approach. Our findings demonstrate that the hwmon-based measurements, with their higher temporal resolution, provide more accurate energy estimates for fine-grained code regions compared to the coarse-grained averaging of NVML. This precision is particularly critical for capturing transient power spikes and characterizing energy consumption during brief but intensive computational phases.

Although our tool set was demonstrated on the GH200 superchip, the approach is architecture agnostic and can be extended to systems such as AMD Instinct or Intel Xe, broadening its applicability to various HPC environments. Future work includes adding more sensor support, exploring power-aware scheduling strategies, investigating the effects of power capping across workloads, and conducting comprehensive studies on various HPC applications to further refine the methodology. Furthermore, improving the

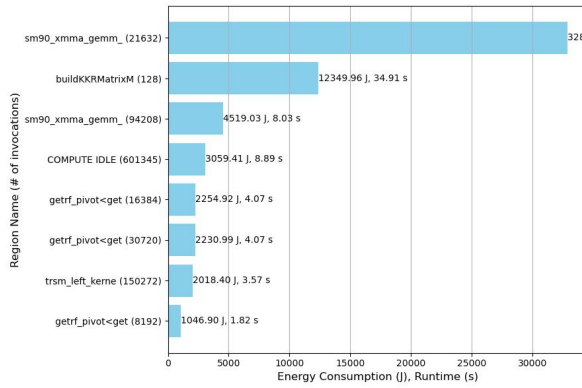
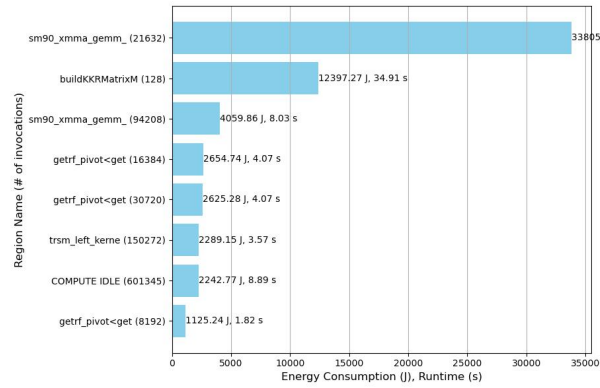
(a) Energy estimation using `nvmlDeviceGetPowerUsage()`(b) Energy estimation using `hwmn` power measurements

Figure 6: Estimated energy (in joules) consumed by the most time-consuming GPU regions executed on the GH200 superchip. The graph shows the region name (e.g., GPU kernel), the number of invocations (in parentheses), the total energy consumed (in joules), and the total time spent per region (in seconds).

integration of Parquet-based trace analytics with AI-driven optimization frameworks presents a promising avenue for automation and scalability for energy-efficient optimizations.

Acknowledgments

This research used resources from the Oak Ridge Leadership Computing Facility, which is a US Department of Energy (DOE) Office of Science user facility supported under contract DE-AC05-00OR22725. This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research under contract number DE-AC05-00OR22725.

References

- [1] 2024. Linaro MAP: High-Performance Profiling and Analysis Tool. <https://www.linaro.org/linaro-map/> Accessed: 2024-12-24.
- [2] AMD. 2024. AMD Omnitrace Documentation. <https://rocm.docs.amd.com/projects/omnitrace/en/latest/doxygen/html/index.html>. <https://rocm.docs.amd.com/projects/omnitrace/en/latest/doxygen/html/index.html> Accessed: 2024-12-25.
- [3] Francesco Antici, Andrea Bartolini, Zeynep Kiziltan, Ozalp Babaoglu, Yuetsu Kodama, et al. 2024. MCBound: An Online Framework to Characterize and Classify Memory/Compute-bound HPC Jobs. In *SC'24: Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. 1–15.
- [4] Scott Atchley, Christopher Zimmer, John Lange, David Bernholdt, Veronica Melles Vergara, Thomas Beck, Michael Brim, Reuben Budiardja, Sunita Chandrasekaran, Markus Eisenbach, Thomas Evans, Matthew Ezell, Nicholas Frontiere, Antigoni Georgiadou, Joe Glenski, Philipp Grete, Steven Hamilton, John Holmen, Axel Huebl, Daniel Jacobson, Wayne Joubert, Kim McMahon, Elia Merzari, Stan Moore, Andrew Myers, Stephen Nichols, Sarp Oral, Thomas Papatheodore, Danny Perez, David M. Rogers, Evan Schneider, Jean-Luc Vay, and P. K. Yeung. 2023. Frontier: Exploring Exascale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Denver, CO, USA) (SC '23). Association for Computing Machinery, New York, NY, USA, Article 52, 16 pages. <https://doi.org/10.1145/3581784.3607089>
- [5] Kendall E. Atkinson. 1989. *An Introduction to Numerical Analysis*. John Wiley & Sons.
- [6] Stefano Corda, Bram Veenboer, and Emma Tolley. 2022. PMT: Power Measurement Toolkit. In *2022 IEEE/ACM International Workshop on HPC User Support Tools (HUST)*. 44–47. <https://doi.org/10.1109/HUST56722.2022.00011>
- [7] NVIDIA Corporation. 2024. NVIDIA Data Center GPU Manager (DCGM). <https://developer.nvidia.com/dcgmm>. <https://developer.nvidia.com/dcgmm> Accessed: 2024-12-25.
- [8] NVIDIA Corporation. 2024. NVML Power and Temperature Metrics - Nsight Systems User Guide. <https://docs.nvidia.com/nsight-systems/UserGuide/index.html#nvml-power-and-temperature-metrics-preview> Accessed: 2024-12-21.
- [9] Robert Dietrich, Frank Winkler, Andreas Knüpfer, and Wolfgang Nagel. 2020. Pika: Center-wide and job-aware cluster monitoring. In *2020 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 424–432.
- [10] Naresh Dulam, Abhilash Katari, and Kishore Reddy Gade. 2017. Apache Arrow: Optimizing Data Interchange in Big Data Systems. *Distributed Learning and Broad Applications in Scientific Research* 3 (2017), 93–114.
- [11] Markus Eisenbach, Jeff Larkin, Justin Lutjens, Steven Rennich, and James H. Rogers. 2017. GPU acceleration of the Locally Selfconsistent Multiple Scattering code for first principles calculation of the ground state and statistical physics of materials. *Computer Physics Communications* 211 (2017), 2–7. <https://doi.org/10.1016/j.cpc.2016.07.013> High Performance Computing for Advanced Modeling and Simulation of Materials.
- [12] Dominic Eschweiler, Michael Wagner, Markus Geimer, Andreas Knüpfer, Wolfgang E Nagel, and Felix Wolf. 2012. Open trace format 2: The next generation of scalable trace formats and support libraries. In *Applications, Tools and Techniques on the Road to Exascale Computing*. IOS Press, 481–490.
- [13] Argonne Leadership Computing Facility. 2024. THAPI: Tracing Heterogeneous APIs. <https://github.com/argonne-lcf/THAPI> Accessed: 2024-12-24.
- [14] Markus Geimer, Felix Wolf, Brian JN Wylie, Erika Abraham, Daniel Becker, and Bernd Mohr. 2010. The Scalasca performance toolset architecture. *Concurrency and computation: Practice and experience* 22, 6 (2010), 702–719.
- [15] Michael Gerndt. 2016. The READEX Project for Dynamic Energy Efficiency Tuning. In *Proceedings of the ACM Workshop on Software Engineering Methods for Parallel and High Performance Applications*. 11–12.
- [16] Heike Jagode, Anthony Danalis, Giuseppe Congiu, Daniel Barry, Anthony Castaldo, and Jack Dongarra. 2024. Advancements of PAPI for the exascale generation. *The International Journal of High Performance Computing Applications* (2024), 10943420241303884.
- [17] Kashif Nizam Khan, Mikael Hirki, Tapio Niemi, Jukka K Nurminen, and Zhonghong Ou. 2018. Rapl in action: Experiences in using rapl for power measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems (TOMPECS)* 3, 2 (2018), 1–26.
- [18] Andreas Knüpfer, Christian Rössel, Dieter an Mey, Scott Biersdorff, Kai Diethelm, Dominic Eschweiler, Markus Geimer, Michael Gerndt, Daniel Lorenz, Allen Malony, Wolfgang E. Nagel, Yury Oleynik, Peter Philippen, Pavel Saviankou, Dirk Schmidl, Sameer Shende, Ronny Tschüter, Michael Wagner, Bert Wesarg, and Felix Wolf. 2012. Score-P: A Joint Performance Measurement Run-Time Infrastructure for Periscope, Scalasca, TAU, and Vampir. In *Tools for High Performance Computing 2011*, Holger Brunst, Matthias S. Müller, Wolfgang E. Nagel, and Michael M. Resch (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 79–91.
- [19] Karlo Kraljic, Daniel Kerger, and Martin Schulz. 2022. Energy Efficient Frequency Scaling on A GPUs in A Heterogeneous HPC Systems. In *Architecture of Computing Systems*, Martin Schulz, Carsten Trinitis, Nikela Papadopoulos, and Thilo Pionteck (Eds.). Springer International Publishing, Cham, 3–16.
- [20] Adam Krzywniak and Pawel Czarnul. 2023. Dynamic GPU Power Capping with Online Performance Tracing. *Future Generation Computer Systems* 145 (2023),

- 396–414.
- [21] Sandia National Laboratories. 2024. *Variorum Documentation*. <https://variorem.readthedocs.io/en/latest/> Accessed: 2024-12-24.
 - [22] Steven J Martin and Matthew Kappel. 2014. Cray XC30 power monitoring and management. In *Cray User Group Conference Proceedings*.
 - [23] Joseph Meyer, Reza Moghimi, and Noah Sturcken. 2019. Package Voltage Regulators: The Answer for Power Management Challenges. *IMAPSource Proceedings* 2019, 1 (oct 1 2019), 438–443. <https://doi.org/10.4071/2380-4505-2019.1.000438>
 - [24] NVIDIA Corporation. 2023. Grace Hopper Superchip Performance Tuning Guide. <https://docs.nvidia.com/grace-perf-tuning-guide/index.html#power-telemetry> Accessed: 2024-12-18.
 - [25] NVIDIA Corporation. 2024. NVIDIA Management Library (NVML) API Documentation: Device Queries. https://docs.nvidia.com/deploy/nvml-api/group_nvmlDeviceQueries.html#group_nvmlDeviceQueries_1g7ef7dff0ff14238d08a19ad7fb23fc87 Accessed: 2024-12-19.
 - [26] NVIDIA Corporation. 2025. *CUDA Profiling Tools Interface (CUPTI) Documentation*. <https://docs.nvidia.com/cupti/index.html>.
 - [27] Pavel Saviankou, Michael Knobloch, Anke Visser, and Bernd Mohr. 2015. Cube v4: From performance report explorer to performance analysis tool. *Procedia Computer Science* 51 (2015), 1343–1352.
 - [28] Robert Schöne, Ronny Tschüter, Thomas Ilsche, Joseph Schuchart, Daniel Hackenberg, and Wolfgang E Nagel. 2017. Extending the functionality of score-p through plugins: Interfaces and use cases. In *Tools for High Performance Computing 2016: Proceedings of the 10th International Workshop on Parallel Tools for High Performance Computing, October 2016, Stuttgart, Germany*. Springer, 59–82.
 - [29] Sameer Shende and Allen D Malony. 2011. TAU: Performance Analysis and Tuning. *Innovative Parallel Computing (InPar)* (2011).
 - [30] Sameer S Shende and Allen D Malony. 2006. The TAU parallel performance system. *The International Journal of High Performance Computing Applications* 20, 2 (2006), 287–311.
 - [31] Osman Seckin Simsek, Jean-Guillaume Piccinali, and Florina M Ciorba. 2023. Accurate Measurement of Application-level Energy Consumption for Energy-Aware Large-Scale Simulations. In *Proceedings of the SC’23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*. 1881–1884.
 - [32] Supermicro. 2025. NVIDIA GH200 Grace Hopper Superchip system supporting NVIDIA BlueField-3 or NVIDIA ConnectX-7. <https://www.supermicro.com/en/products/system/gpu/1u/ars-111gl-nhr> Accessed: 2025-01-13.
 - [33] Vince Weaver. 2012. New features in the PAPI 5.0 release. *University of Tennessee, Tech. Rep* (2012).
 - [34] Matthias Weber, Ronny Brendel, Michael Wagner, Robert Dietrich, Ronny Tschüter, and Holger Brunst. 2019. Visual Comparison of Trace Files in Vampir. In *Programming and Performance Visualization Tools*, Abhinav Bhatele, David Boehme, Joshua A. Levine, Allen D. Malony, and Martin Schulz (Eds.). Springer International Publishing, Cham, 105–121.
 - [35] Zeyu Yang, Karel Adamek, and Wesley Armour. 2024. Accurate and Convenient Energy Measurements for GPUs: A Detailed Study of NVIDIA GPU’s Built-In Power Sensor (SC ’24). IEEE Press, Article 22, 17 pages. <https://doi.org/10.1109/SC41406.2024.00028>